



Mark Litwintschik

I'm a Big Data, AI, GIS & Networking Consultant with clients in the UK, USA, Sweden, Ireland & Germany. Past clients include BAA plc, Bank of America Merrill Lynch, Blackberry, Bloomberg, British Telecom, Ford, Google, ITV, IMG, Nando's, News UK, Pizza Hut, Royal Mail, T-Mobile, Williams Formula 1, Wise & UBS. I hold both a Canadian and a British passport as well as permanent residence in Estonia. Find me on [LinkedIn](#) & [X](#).

[HOME](#) | [BENCHMARKS](#) | [CATEGORIES](#) | [ATOM FEED](#)

POSTED ON FRI 15 MARCH 2024 UNDER [DATABASES](#)

1.1 Billion Taxi Rides using DuckDB

DuckDB is an in-process database. Rather than relying on a server of its own, it's used as a client. The client can work with data in memory, within DuckDB's internal file format, database servers from other software developers and cloud storage services such as AWS S3.

This choice to not centralise DuckDB's data within its own server, paired with being distributed as a single binary, makes installing and working with DuckDB much less complex than say, standing up a Hadoop Cluster.

The project isn't aimed at very large datasets. Despite this, its ergonomics are enticing enough and it does so much to reduce engineering time that workarounds are worth considering. The rising popularity of analysis-ready, cloud-optimised Parquet files is removing the need for substantial hardware when dealing with datasets in the 100s of GBs or larger.

DuckDB is primarily the work of [Mark Raasveldt](#) and [Hannes Mühleisen](#). It's made up of a million lines of C++ and runs as a stand-alone binary. Development is very active with the commit count on its GitHub repo doubling nearly every year since it began in 2018. DuckDB uses PostgreSQL's SQL parser, Google's RE2 regular expression engine and SQLite's shell.

SQLite supports five data types, NULL, INTEGER, REAL, TEXT and BLOB. I was always frustrated by this as working with time would require transforms in every SELECT statement and not being able to describe a field as a boolean meant analysis software couldn't automatically recognise and provide specific UI controls and visualisations of these fields.

Thankfully, DuckDB supports [25 data types](#) out of the box and more can be added via extensions.

A min-max index is created for every column segment in DuckDB. This index type is how most OLAP database engines answer aggregate queries so quickly. Parquet and JSON extensions are shipped in the official build and their usage is well documented. Both Snappy and ZStandard compression are supported for Parquet files.

DuckDB's documentation is well-organised and refreshingly terse with [examples](#) next to most descriptions.

In this post, I'm going to see how fast DuckDB can run the 1.1 billion taxi rides benchmark. This dataset is made up of 1.1 billion taxi trips conducted in New York City between 2009 and 2015.

This is the same dataset I've used to benchmark Amazon Athena, BigQuery, BrytlytDB, ClickHouse, Elasticsearch, EMR, Hydrolix, kdb+/q, MapD / OmniSci / HEAVY.AI, PostgreSQL, Redshift and Vertica. I have a [single-page summary](#) of all these benchmarks for comparison.

My Workstation

For this benchmark, I'm using a 6 GHz Intel Core i9-14900K CPU. It has 8 performance cores and 16 efficiency cores with a total of 32 threads and 32 MB of L2 cache. It has a liquid cooler attached and is housed in a spacious, full-sized, Cooler Master HAF 700 computer case. I've come across videos on YouTube where people have managed to overclock the i9-14900KF to 9.1 GHz.

The system has 48 GB of DDR5 RAM clocked at 5,200 MHz and a 5th-generation, Crucial T700 4 TB NVMe M.2 SSD which can read at speeds up to 12,400 MB/s. There is a heatsink on the SSD to help keep its temperature down. This is my system's C drive.

There is also a 2 TB SSD connected via a SATA interface which contains the original taxi rides dataset. This drive peaks around 5-600 MB/s. This is my system's D drive.

The system is powered by a 1,200-watt, fully modular, Corsair Power Supply and is sat on an ASRock Z790 Pro RS Motherboard.

I'm running Ubuntu 22 LTS via Microsoft's Ubuntu for Windows on Windows 11 Pro. In case you're wondering why I don't run a Linux-based desktop as my primary work environment, I'm still using an Nvidia GTX 1080 GPU which has better driver support on Windows and I use ArcGIS Pro from time to time which only supports Windows natively.

DuckDB Up & Running

I'll first install some dependencies that will be used in this post.

```
$ sudo apt update
$ sudo apt install \
  build-essential \
  cmake \
  pigz \
  python3-virtualenv \
  zip
```

I'll then download and install the official binary for DuckDB v0.10.0 below.

```
$ cd ~
$ wget -c https://github.com/duckdb/duckdb/releases/download/v0.10.0/duckdb_cli-linux-amd64.zip
$ unzip -j duckdb_cli-linux-amd64.zip
$ chmod +x duckdb
$ ~/duckdb
```

```
INSTALL parquet;
```

```
$ vi ~/.duckdbrc
```

```
.timer on
.width 180
LOAD parquet;
```

Importing 1.1 Billion Trips into DuckDB

The dataset I'll be using is a data dump I've produced of 1.1 billion taxi trips conducted in New York City over six years. The raw dataset lives as 56 GZIP-compressed CSV files that are 104 GB when compressed and need 500 GB of space when decompressed. The [Billion Taxi Rides in Redshift](#) blog post goes into detail regarding how I put this dataset together and describes the columns it contains in further detail.

I'll read the source data from `/mnt/d/taxi` which is on the SATA-connected SSD and create a DuckDB file `/mnt/c/taxi/taxi.duckdb` on the 5th-generation NVMe drive.

```
$ cd /mnt/d/taxi/
$ vi create.sql
```

Rather than rely on DuckDB's type inference, I'll create a table for

the data ahead of time. This will ensure I have the right granularity of data types for each column.

```
CREATE OR REPLACE TABLE trips (  
  trip_id          BIGINT,  
  vendor_id        VARCHAR,  
  
  pickup_datetime  TIMESTAMP,  
  dropoff_datetime TIMESTAMP,  
  
  store_and_fwd_flag VARCHAR,  
  rate_code_id      BIGINT,  
  pickup_longitude  DOUBLE,  
  pickup_latitude   DOUBLE,  
  dropoff_longitude DOUBLE,  
  dropoff_latitude  DOUBLE,  
  passenger_count   BIGINT,  
  trip_distance     DOUBLE,  
  fare_amount       DOUBLE,  
  extra             DOUBLE,  
  mta_tax           DOUBLE,  
  tip_amount        DOUBLE,  
  tolls_amount      DOUBLE,  
  ehail_fee         DOUBLE,  
  improvement_surcharge DOUBLE,  
  total_amount      DOUBLE,  
  payment_type      VARCHAR,  
  trip_type         VARCHAR,  
  pickup           VARCHAR,  
  dropoff          VARCHAR,  
  
  cab_type          VARCHAR,  
  
  precipitation     BIGINT,  
  snow_depth        BIGINT,  
  snowfall          BIGINT,  
  max_temperature   BIGINT,  
  min_temperature   BIGINT,  
  average_wind_speed BIGINT,  
  
  pickup_nyct2010_gid BIGINT,  
  pickup_ctlabel      VARCHAR,  
  pickup_borocode     BIGINT,  
  pickup_boroname     VARCHAR,  
  pickup_ct2010       VARCHAR,  
  pickup_boroct2010   BIGINT,  
  pickup_cdeligibil   VARCHAR,  
  pickup_ntacode      VARCHAR,  
  pickup_ntaname      VARCHAR,  
  pickup_puma         VARCHAR,  
  
  dropoff_nyct2010_gid BIGINT,  
  dropoff_ctlabel     VARCHAR,  
  dropoff_borocode    BIGINT,  
  dropoff_boroname    VARCHAR,  
  dropoff_ct2010      VARCHAR,  
  dropoff_boroct2010  VARCHAR,  
  dropoff_cdeligibil  VARCHAR,  
  dropoff_ntacode     VARCHAR,  
  dropoff_ntaname     VARCHAR,  
  dropoff_puma        VARCHAR);
```

```
$ ~/duckdb /mnt/c/taxi/taxi.duckdb < create.sql  
$ ~/duckdb /mnt/c/taxi/taxi.duckdb
```

```
INSERT INTO trips  
SELECT *  
FROM read_csv('trips_x*.csv.gz');
```

Reads peaked at 60 MB/s but sat around ~40 MB/s and writes sat around ~20 MB/s on the SATA drive. CPU usage sat around 25% and DuckDB's RAM consumption quickly grew to around 22 GB. At one point, I could see I was about to run out of RAM so I cancelled the job.

One CSV At A Time

My second attempt at importing the CSVs into DuckDB was to import one CSV file at a time.

```
$ for FILENAME in trips_x*.csv.gz; do
  echo $FILENAME
  ~/duckdb -c "INSERT INTO trips
               SELECT *
               FROM   READ_CSV('$FILENAME');" \
               /mnt/c/taxi/taxi.duckdb
done
```

Initially, this worked well. Reads peaked at 60 MB/s but sat around ~40 MB/s and writes sat around ~20 MB/s on the SATA drive. DuckDB's RAM consumption peaked at around 10 GB during each import. CPU usage initially sat around 15-25% but eventually hit 100% as more and more CSVs were imported into DuckDB.

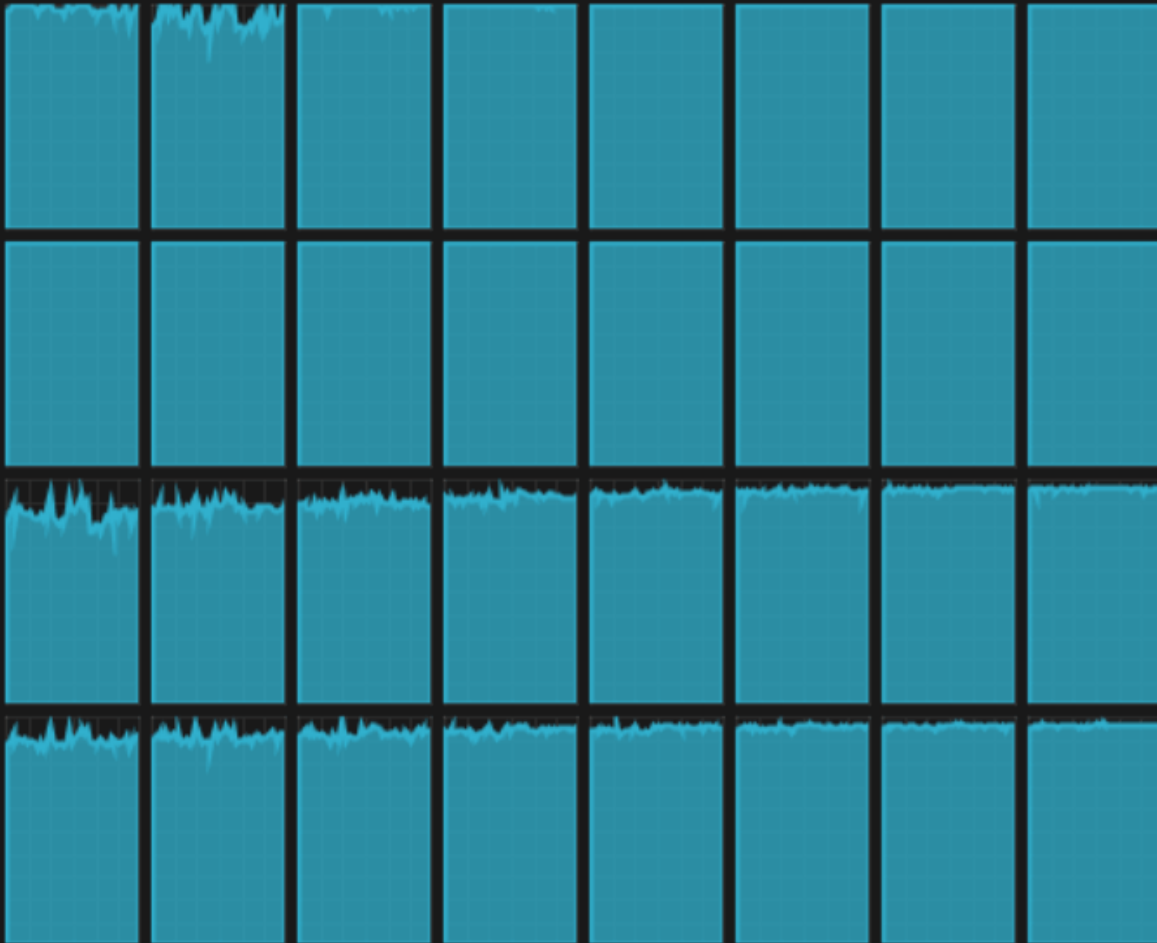
When CPU consumption hit 100%, I noticed the overall clock rate dropped to 3.5 GHz and then again to 3.4 GHz before recovering into the 4 GHz range.

CPU

Intel(R) Core(TM) i9-14900K

% Utilisation over 60 seconds

100%



Utilisation	Speed		Base speed:	3.20 GHz
100%	3.38 GHz		Sockets:	1
			Cores:	24
Processes	Threads	Handles	Logical processors:	32
305	5858	150462	Virtualisation:	Enabled
Up time			L1 cache:	2.1 MB
0:12:47:22			L2 cache:	32.0 MB
			L3 cache:	36.0 MB

Below is a screenshot from Speccy from around the same time.

 CPU

▼ Intel Processor

Cores24

Threads32

NameIntel Processor

SpecificationIntel Core i9-14900K

Family6

Extended Family6

Model7

Extended ModelB7

Stepping1

InstructionsMMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, Intel 64, NX, AES, AVX, AVX2, FMA3

VirtualizationNot supported

HyperthreadingSupported, Disabled

Fan Speed2848 RPM

▼ Caches

L1 Data Cache Size24 x 48 KBytes

L1 Instructions Cache Size24 x 32 KBytes

L2 Unified Cache Size12 x 2048 KBytes

L3 Unified Cache Size36864 KBytes

▼ Cores

	Core Speed	Multiplier	Bus Speed	Threads
Core 0	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 0, 1
Core 1	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 8, 9
Core 2	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 16, 17
Core 3	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 24, 25
Core 4	3177.4 MHz	x 88.5	35.9 MHz 	APIC ID: 32, 33
Core 5	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 40, 41
Core 6	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 48, 49
Core 7	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 56, 57
Core 8	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 64
Core 9	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 66
Core 10	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 68
Core 11	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 70
Core 12	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 72
Core 13	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 74
Core 14	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 76
Core 15	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 78
Core 16	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 80
Core 17	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 82
Core 18	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 84
Core 19	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 86
Core 20	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 88
Core 21	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 90
Core 22	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 92
Core 23	3195.4 MHz	x 89.0	35.9 MHz 	APIC ID: 94

The import progressed well until the 36th CSV file but from there, progress halted with the CPU sat at 100%. The DuckDB file was around 70 GB at this point. After 45 minutes of very little disk activity and the CPU running at full throttle, I cancelled the job.

Building Parquet Files

My third attempt at benchmarking this dataset was to just convert each CSV file into a Parquet file and then query those.

SSDs are consumables and at some point, can be expected to fail given enough usage. The 5th generation SSD cost me €616 whereas a new SATA-based SSD with 2 TB of capacity runs around ~€100. For this reason, I'll run the benchmarks on the faster drive but I'll build the Parquet files on the cheaper drive.

Normally I sort data by longitude as it results in the smaller Parquet files but to keep RAM consumption at a minimum, I'll not specify any particular order to DuckDB.

```
$ for FILENAME in trips_x*.csv.gz; do
  echo $FILENAME

  OUT=`echo $FILENAME | sed 's/.csv.gz//g'`

  touch working.duckdb
  rm working.duckdb

  ~/duckdb working.duckdb < create.sql
  ~/duckdb -c "INSERT INTO trips
              SELECT *
              FROM read_csv('$FILENAME');

              SET preserve_insertion_order=false;
              SET memory_limit='8GB';

              COPY(
                SELECT *
                FROM trips
              ) TO '$OUT.pq' (FORMAT 'PARQUET',
                             CODEC 'ZSTD',
                             ROW_GROUP_SIZE 15000);" \
    working.duckdb
done
```

The above peaked at around 20 GB in RAM consumption. The CSVs read anywhere between 40-120 MB/s and writes peaked around 120 MB/s at any one time. CPU consumption barely broke 20% during certain operations and would peak at 100% across all cores during other parts.

Most of the GZIP-compressed CSV files are 1.9 GB in size and the resulting PQ files are around 1.5 GB.

One Problematic CSV

Unfortunately, DuckDB got stuck on `trips_xbj.csv.gz`. The CPU was maxed out with a working DuckDB file stuck at 868 MB of the ~2 GB that would normally be generated. I suspected the CSV was too large so I tried splitting it up into several 500K-line CSV files.

```
$ pigz -dc trips_xbj.csv.gz \  
  | split --lines=500000 \  
    --filter="pigz > trips_xbj_\\$FILE.csv.gz"  
  
$ for FILENAME in trips_xbj_x*.csv.gz; do  
  echo $FILENAME  
  
  OUT=`echo $FILENAME | sed 's/.csv.gz//g'`  
  
  touch working.duckdb  
  rm working.duckdb  
  
  ~/duckdb working.duckdb < create.sql  
  ~/duckdb -c "INSERT INTO trips  
    SELECT *  
    FROM read_csv('$FILENAME');  
  
    SET preserve_insertion_order=false;  
    SET memory_limit='8GB';  
  
    COPY(  
      SELECT *  
      FROM trips  
    ) TO '$OUT.pq' (FORMAT 'PARQUET',  
      CODEC 'ZSTD',  
      ROW_GROUP_SIZE 15000);" \  
  
  working.duckdb  
  
done
```

When I ran the above DuckDB ended up getting stuck on `trips_xbj_xas.csv.gz` with the same symptoms as before.

Compiling DuckDB

I decided to compile the main branch of DuckDB and see if it could process the above CSV without issue.

```
$ git clone https://github.com/duckdb/duckdb.git ~/duckdb_source
$ cd ~/duckdb_source

$ mkdir -p build/release

$ cmake \
  ./CMakeLists.txt \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo \
  -DEXTENSION_STATIC_BUILD=1 \
  -DBUILD_PARQUET_EXTENSION=1 \
  -B build/release

$ CMAKE_BUILD_PARALLEL_LEVEL=$(nproc) \
  cmake --build build/release
```

I re-installed the Parquet extension for this distinct DuckDB build.

```
$ ~/duckdb_source/build/release/duckdb
```

```
INSTALL parquet;
```

I then re-ran the CSV to Parquet conversion process on the problematic CSV.

```
$ cd /mnt/d/taxi

$ for FILENAME in trips_xbj_xas.csv.gz; do
  echo $FILENAME

  OUT=`echo $FILENAME | sed 's/.csv.gz//g'`

  touch working.duckdb
  rm working.duckdb

  ~/duckdb_source/build/release/duckdb working.duckdb < create.sql
  ~/duckdb_source/build/release/duckdb \
    -c "INSERT INTO trips
        SELECT *
        FROM read_csv('$FILENAME');

        SET preserve_insertion_order=false;
        SET memory_limit='8GB';

        COPY(
          SELECT *
          FROM trips
        ) TO '$OUT.pq' (FORMAT 'PARQUET',
                        CODEC 'ZSTD',
                        ROW_GROUP_SIZE 15000);" \
    working.duckdb
done
```

I was hoping that there was some sort of CSV parsing bug that had been fixed sometime in the past few weeks since the 0.10.0 release that might resolve the issue. Instead, I got an error message that I hadn't seen with the official binary release.

```
Conversion Error: CSV Error on Line: 220725
Error when converting column "column20".
Could not convert string "Cash" to 'BIGINT'
```

```
file=trips_xbj_xas.csv.gz
delimiter = , (Auto-Detected)
quote = \" (Auto-Detected)
escape = \" (Auto-Detected)
new_line = \" (Auto-Detected)
header = false (Auto-Detected)
skip_rows = 0 (Auto-Detected)
date_format = (Auto-Detected)
timestamp_format = (Auto-Detected)
null_padding=0
sample_size=20480
ignore_errors=0
all_varchar=0
```

The column names start from column00. The 21st field is payment_type which is a VARCHAR. I'm not sure why DuckDB would want to cast it to a BIGINT.

I've removed the readability spacing between the fields in create.sql. The first row is the CREATE TABLE statement so the 22nd line is the 21st field name and type.

```
$ grep -n -B1 -A1 payment create.sql
```

```
21-         total_amount          DOUBLE,
22-         payment_type          VARCHAR,
23-         trip_type              VARCHAR,
```

The field is a VARCHAR so I'm confused as to why DuckDB would want to cast it to a BIGINT.

Ignoring Errors

I'll run the CSV to Parquet conversion again with ignore_errors=true to see if I can get past this record and see if I can get the scope of how widespread this issue is.

```
$ for FILENAME in trips_xbj_xas.csv.gz; do
  echo $FILENAME

  OUT=`echo $FILENAME | sed 's/.csv.gz//g'`

  touch working.duckdb
  rm working.duckdb

  ~/duckdb_source/build/release/duckdb working.duckdb < create.sql
  ~/duckdb_source/build/release/duckdb \
    -c "INSERT INTO trips
        SELECT *
        FROM read_csv('$FILENAME',
                      ignore_errors=true);"
```

```

SET preserve_insertion_order=false;
SET memory_limit='8GB';

COPY(
  SELECT *
  FROM trips
) TO '$OUT.pq' (FORMAT 'PARQUET',
               CODEC 'ZSTD',
               ROW_GROUP_SIZE 15000);" \

```

working.duckdb

done

There should be 500K records in the resulting Parquet file but there aren't even 280K.

```

SELECT COUNT(*)
FROM READ_PARQUET('trips_xbj_xas.pq');

```

count_star() int64
279904

I also found both `trips_xbs.pq` and `trips_xbv.pq` were collectively missing around 2M records as well.

```

$ for FILENAME in trips_x*.pq; do
  echo $FILENAME, `~/duckdb -csv -c "SELECT COUNT(*) FROM READ_PARQUET('$FILE'"
done

```

```

...
trips_xbs.pq, .. count_star() 19999999
trips_xbt.pq, .. count_star() 20000000
trips_xbu.pq, .. count_star() 20000000
trips_xbv.pq, .. count_star() 17874858
...

```

Troublesome Records

This is the portion of the record DuckDB was complaining about.

```

$ gunzip -c trips_xbj_xas.csv.gz \
  | head -n220725 \
  | tail -n1 \
  > unparsable.csv
$ grep -o '.*Cash' unparsable.csv

```

```

26948130,CMT,2009-01-10 11:24:50,2009-01-10 11:28:39,,,-73.980964,40.77938799

```

Below I'll use DuckDB to read the record in and see which field the

'Cash' value lands on.

```
$ ~/duckdb
```

```
.mode line
```

```
SELECT COLUMNS(c -> c LIKE 'column0%' OR  
                    c LIKE 'column1%' or  
                    c LIKE 'column20')  
FROM READ_CSV('unparsable.csv');
```

```
column00 = 26948130  
column01 = CMT  
column02 = 2009-01-10 11:24:50  
column03 = 2009-01-10 11:28:39  
column04 =  
column05 =  
column06 = -73.980964  
column07 = 40.779388  
column08 = -73.97072  
column09 = 40.784226  
column10 = 1  
column11 = 0.9  
column12 = 4.5  
column13 = 0  
column14 =  
column15 = 0  
column16 = 0  
column17 =  
column18 =  
column19 = 4.5  
column20 = Cash
```

The 'Cash' value is on the 21st column so this doesn't look like an alignment issue.

The CSV file can be parsed by a comma delimiter alone as none of the cell values contain commas. Below I'll annotate the first 21 rows of the CSV data with a row number as a double check that 'Cash' should appear in the 21st column.

```
$ python3
```

```
s = '''26948130,CMT,2009-01-10 11:24:50,2009-01-10 11:28:39,,,-73.980964,40.77  
for n, v in enumerate(s.split(','), start=1):  
    print(n, v)
```

```
1 26948130  
2 CMT  
3 2009-01-10 11:24:50  
4 2009-01-10 11:28:39  
5  
6  
7 -73.980964  
8 40.779387999999997  
9 -73.97072  
10 40.784225999999997  
11 1
```

```
12 0.900000000000000002
13 4.5
14 0
15
16 0
17 0
18
19
20 4.5
21 Cash
```

Converting with ClickHouse

I'm not sure how to resolve the issue with DuckDB in a timely manner. As a workaround, I'll use ClickHouse to produce the remaining Parquet files.

```
$ cd ~
$ curl https://clickhouse.com/ | sh
$ cd /mnt/d/taxi
$ ~/clickhouse local
```

```
CREATE TABLE trips (
    trip_id          INT,
    vendor_id        VARCHAR(3),
    pickup_datetime  TIMESTAMP,
    dropoff_datetime TIMESTAMP,
    store_and_fwd_flag VARCHAR(1),
    rate_code_id     SMALLINT,
    pickup_longitude DECIMAL(18,14),
    pickup_latitude  DECIMAL(18,14),
    dropoff_longitude DECIMAL(18,14),
    dropoff_latitude DECIMAL(18,14),
    passenger_count  SMALLINT,
    trip_distance    DECIMAL(18,6),
    fare_amount      DECIMAL(18,6),
    extra            DECIMAL(18,6),
    mta_tax          DECIMAL(18,6),
    tip_amount       DECIMAL(18,6),
    tolls_amount     DECIMAL(18,6),
    ehail_fee        DECIMAL(18,6),
    improvement_surcharge DECIMAL(18,6),
    total_amount     DECIMAL(18,6),
    payment_type     VARCHAR(3),
    trip_type        SMALLINT,
    pickup           VARCHAR(50),
    dropoff          VARCHAR(50),

    cab_type         VARCHAR(6),

    precipitation    SMALLINT,
    snow_depth       SMALLINT,
    snowfall         SMALLINT,
    max_temperature  SMALLINT,
    min_temperature  SMALLINT,
    average_wind_speed SMALLINT,

    pickup_nyct2010_gid SMALLINT,
    pickup_ctlabel      VARCHAR(10),
    pickup_borocode     SMALLINT,
    pickup_boroname     VARCHAR(13),
    pickup_ct2010       VARCHAR(6),
    pickup_boroc2010    VARCHAR(7),
```

```

pickup_cdeligibil    VARCHAR(1),
pickup_ntacode       VARCHAR(4),
pickup_ntaname       VARCHAR(56),
pickup_puma          VARCHAR(4),

dropoff_nyct2010_gid  SMALLINT,
dropoff_ctlabel       VARCHAR(10),
dropoff_borocode     SMALLINT,
dropoff_boroname      VARCHAR(13),
dropoff_ct2010       VARCHAR(6),
dropoff_boroct2010   VARCHAR(7),
dropoff_cdeligibil   VARCHAR(1),
dropoff_ntacode       VARCHAR(4),
dropoff_ntaname       VARCHAR(56),
dropoff_puma         VARCHAR(4)
) ENGINE=Log;

INSERT INTO trips
SELECT *
FROM file('trips_xbj_xas.csv.gz', CSV);

SELECT *
FROM trips
INTO OUTFILE 'trips_xbj_xas.pq'
FORMAT Parquet;

```

```

500000 rows in set. Elapsed: 2.427 sec. Processed 500.00 thousand rows, 290.10
Peak memory usage: 418.85 MiB.

```

I tried to get DuckDB to read the above Parquet file but it wasn't able to.

```
$ ~/duckdb
```

```

SELECT    cab_type,
          COUNT(*)
FROM      READ_PARQUET('trips_xbj_xas.pq')
GROUP BY cab_type;

```

```
Error: Invalid Error: Unsupported compression codec "7". Supported options are
```

Examining ClickHouse's Parquet File

I've been working on a [Parquet debugging tool](#). I'll use it to examine the compression scheme ClickHouse is using.

```

$ git clone https://github.com/marklit/pqview \
  ~/pqview

$ virtualenv ~/.pqview
$ source ~/.pqview/bin/activate
$ python3 -m pip install \
  -r ~/pqview/requirements.txt

```

```

$ python3 ~/pqview/main.py \
  most-compressed \
  trips_xbj_xas.pq \

```

```
| grep compression
```

```
compression: LZ4
```

ClickHouse changed its default Parquet compression scheme from Snappy to LZ4 at some point last year. LZ4 is great but DuckDB doesn't support it. I'll rebuild the Parquet file using a ClickHouse compatibility setting that'll revert to using Snappy compression instead.

```
SELECT *
FROM trips
INTO OUTFILE 'trips_xbj_xas.pq'
FORMAT Parquet
SETTINGS compatibility='23.2';
```

```
$ python3 ~/pqview/main.py \
    most-compressed \
    trips_xbj_xas.pq \
    | grep compression
```

```
compression: SNAPPY
```

With ClickHouse able to produce the Parquet files using Snappy, I also re-processed I both `trips_xbs.pq` and `trips_xbv.pq`. I had issues importing them into ClickHouse with `trips_xbv.pq` stalling at the 15.73M-row point. I had to break them both into 4 x 5M-line CSV files to get them imported without issue.

ClickHouse exports timestamps as uint64s to Parquet instead of timestamps. I used DuckDB to convert these fields back into timestamps. Please excuse the lack of a for loop.

```
$ ~/duckdb_source/build/release/duckdb working.duckdb
```

```
SET preserve_insertion_order=false;
SET memory_limit='8GB';

CREATE OR REPLACE TABLE trips AS
  SELECT * EXCLUDE (pickup_datetime, dropoff_datetime),
    MAKE_TIMESTAMP(pickup_datetime * 1000000) AS pickup_datetime,
    MAKE_TIMESTAMP(dropoff_datetime * 1000000) AS dropoff_datetime,
  FROM READ_PARQUET('trips_xbj_xas.pq');

COPY(
  SELECT *
  FROM trips
) TO 'trips_xbj_xas_ts.pq' (FORMAT 'PARQUET',
  CODEC 'ZSTD',
  ROW_GROUP_SIZE 15000);

CREATE OR REPLACE TABLE trips AS
```

```

SELECT * EXCLUDE (pickup_datetime, dropoff_datetime),
        MAKE_TIMESTAMP(pickup_datetime * 1000000) AS pickup_datetime,
        MAKE_TIMESTAMP(dropoff_datetime * 1000000) AS dropoff_datetime,
FROM READ_PARQUET('trips_xbs.pq');

COPY(
  SELECT *
  FROM trips
) TO 'trips_xbs_ts.pq' (FORMAT 'PARQUET',
                      CODEC 'ZSTD',
                      ROW_GROUP_SIZE 15000);

CREATE OR REPLACE TABLE trips AS
  SELECT * EXCLUDE (pickup_datetime, dropoff_datetime),
        MAKE_TIMESTAMP(pickup_datetime * 1000000) AS pickup_datetime,
        MAKE_TIMESTAMP(dropoff_datetime * 1000000) AS dropoff_datetime,
FROM READ_PARQUET('trips_xbv.pq');

COPY(
  SELECT *
  FROM trips
) TO 'trips_xbv_ts.pq' (FORMAT 'PARQUET',
                      CODEC 'ZSTD',
                      ROW_GROUP_SIZE 15000);

```

Double Checking

There should be 1,086,709,191 yellow cab records and 26,943,827 green cab records in a complete 1.1 billion taxi rides dataset. I ran the following to check that this was the case with these Parquet files. Mark Raasveldt fixed a [progress bar bug](#) a few days ago so I'll use the newly compiled version of DuckDB from here on.

```
$ ~/duckdb_source/build/release/duckdb
```

```

SELECT  cab_type,
        COUNT(*)
FROM    READ_PARQUET('trips_x*.pq')
GROUP BY cab_type;

```

cab_type varchar	count_star() int64
yellow	1086709191
green	26943827

The Parquet Benchmark

I copied the Parquet files onto the 5th-generation SSD before

running the following benchmark. The following were the fastest times I saw after running each query multiple times on the Parquet files.

The following completed in 36.942 seconds.

```
SELECT    cab_type,
          COUNT(*)
FROM      READ_PARQUET('trips_x*.pq')
GROUP BY cab_type;
```

The following completed in 51.085 seconds.

```
SELECT    passenger_count,
          AVG(total_amount)
FROM      READ_PARQUET('trips_x*.pq')
GROUP BY passenger_count;
```

The following completed in 60.656 seconds.

```
SELECT    passenger_count,
          DATE_PART('year', pickup_datetime) AS year,
          COUNT(*)
FROM      READ_PARQUET('trips_x*.pq')
GROUP BY passenger_count,
          year;
```

The following completed in 92.124 seconds.

```
SELECT    passenger_count,
          DATE_PART('year', pickup_datetime) AS year,
          ROUND(trip_distance) AS distance,
          COUNT(*)
FROM      READ_PARQUET('trips_x*.pq')
GROUP BY passenger_count,
          year,
          distance
ORDER BY year,
          count(*) DESC;
```

An Alternative Code Path

After the initial publication of this post, Mark Raasveldt reached out and let me know that the COPY command uses an alternative code path to READ_CSV() and that loading the dataset into DuckDB's internal format via this method would be both more reliable and result in faster benchmark numbers.

I removed any .csv.gz files from the SATA-connected SSD that weren't in the original dataset and made sure only the 56 original files remained to avoid record duplication. I then ran the following.

```
$ touch working.duckdb
$ rm working.duckdb

$ ~/duckdb_source/build/release/duckdb \
  working.duckdb \
  < create.sql

$ ~/duckdb_source/build/release/duckdb \
  working.duckdb \
  -c "COPY trips FROM 'trips*.csv.gz';"
```

RAM consumption quickly grew to 22 GB before levelling off. The SSD was writing at 50-150 MB/s and usually reading at 5-10 MB/s while the CPU was almost idle at 5-10%. The job ran for ~3-4 hours before reaching 91% and then abruptly being killed. I'm not sure why this happened as there was ample RAM available on the system.

During the ~3.5 hours there were sustained writes of ~70 MB/s on average which means the SSD would have had ~860 GB written to it despite the DuckDB file only growing to ~90 GB. Given SSDs can wear out this is something to take into consideration if you're using your own hardware.

I made a second attempt at using the COPY command. This time, I imported files one at a time. This job ended up finishing in 75 minutes. The RAM consumption only peaked around 6 GB and the CPU hovered between 15-30% during this time.

```
$ touch working.duckdb
$ rm working.duckdb

$ ~/duckdb_source/build/release/duckdb \
  working.duckdb \
  < create.sql

$ for FILENAME in trips_*.csv.gz; do
  echo `date`, $FILENAME
  ~/duckdb_source/build/release/duckdb \
    -c "COPY trips FROM '$FILENAME';" \
    working.duckdb
done
```

The resulting DuckDB file was 113 GB. I checked the record count

before copying it over to the 5th-generation SSD.

```
SELECT cab_type,
COUNT(*)
FROM trips
GROUP BY cab_type;
```

cab_type varchar	count_star() int64
green	26943827
yellow	1086709191

One interesting aspect of DuckDB is its wide variety of compression schemes. Some of these, like Adaptive Lossless Floating-Point Compression (ALP), are unique to DuckDB. Any one field can have any number of these schemes used to compress their individual row-groups. DuckDB decides automatically which scheme to use on a per-row-group-basis.

Below are the row-group counts for each compress scheme for each of the fields in the trips table.

```
WITH pivot_alias AS (
  PIVOT PRAGMA_STORAGE_INFO('trips')
  ON compression
  USING COUNT(*)
  GROUP BY column_name,
  column_id
  ORDER BY column_id
)
SELECT * EXCLUDE(column_id)
FROM pivot_alias;
```

column_name varchar	ALP int64	ALPRD int64	BitPacking int64	Constant int64	Dictionary int64
trip_id	0	0	9426	9426	0
vendor_id	0	0	0	9426	9425
pickup_datetime	0	0	28218	9426	0
dropoff_datetime	0	0	28218	9426	0
store_and_fwd_flag	0	0	0	888	9283
rate_code_id	0	0	439	10965	0
pickup_longitude	18636	0	0	0	0
pickup_latitude	15872	2663	0	0	0
dropoff_longitude	19353	0	0	0	0
dropoff_latitude	16789	2663	0	0	0
passenger_count	0	0	9426	9426	0
trip_distance	9426	0	0	9426	0
fare_amount	9426	0	0	9426	0
extra	9426	0	0	9426	0
mta_tax	197	0	0	10085	0
tip_amount	9426	0	0	9426	0
tolls_amount	5907	0	0	9426	0
ehail_fee	0	0	0	18852	0
improvement_surcharge	192	0	0	18102	0

total_amount	9427	0	0	9426	0
payment_type	0	0	0	9426	9425
trip_type	0	0	0	9340	9416
pickup	0	0	0	0	8798
dropoff	0	0	0	0	8311
cab_type	0	0	0	9426	9426
precipitation	0	0	9426	9426	0
snow_depth	0	0	2362	16249	0
snowfall	0	0	2262	15614	0
max_temperature	0	0	9426	9426	0
min_temperature	0	0	9426	9426	0
average_wind_speed	0	0	9426	9426	0
pickup_nyct2010_gid	0	0	9426	0	0
pickup_ctlabel	0	0	0	0	7685
pickup_borocode	0	0	9426	0	0
pickup_boroname	0	0	0	0	9426
pickup_ct2010	0	0	0	0	9323
pickup_boroc2010	0	0	18354	0	0
pickup_cdeligibil	0	0	0	0	9426
pickup_ntacode	0	0	0	0	9403
pickup_ntaname	0	0	0	0	9426
pickup_puma	0	0	0	0	9424
dropoff_nyct2010_gid	0	0	647	0	0
dropoff_ctlabel	0	0	0	0	6843
dropoff_borocode	0	0	9426	0	0
dropoff_boroname	0	0	0	0	9426
dropoff_ct2010	0	0	0	0	9221
dropoff_boroc2010	0	0	0	0	9301
dropoff_cdeligibil	0	0	0	0	9426
dropoff_ntacode	0	0	0	0	9407
dropoff_ntaname	0	0	0	0	9426
dropoff_puma	0	0	0	0	9421

51 rows

The Internal Format Benchmark

The following were the fastest times I saw after running each query multiple times on the DuckDB table.

The following completed in 0.498 seconds.

```
SELECT cab_type,
COUNT(*)
FROM trips
GROUP BY cab_type;
```

The following completed in 0.234 seconds.

```
SELECT passenger_count,
AVG(total_amount)
FROM trips
GROUP BY passenger_count;
```

The following completed in 0.734 seconds.

```
SELECT passenger_count,
```

```
DATE_PART('year', pickup_datetime) AS year,  
COUNT(*)  
FROM trips  
GROUP BY passenger_count,  
year;
```

The following completed in 1.334 seconds.

```
SELECT passenger_count,  
DATE_PART('year', pickup_datetime) AS year,  
ROUND(trip_distance) AS distance,  
COUNT(*)  
FROM trips  
GROUP BY passenger_count,  
year,  
distance  
ORDER BY year,  
count(*) DESC;
```

Thank you for taking the time to read this post. I offer both consulting and hands-on development services to clients in North America and Europe. If you'd like to discuss how my offerings can help your business please contact me via [LinkedIn](#).

[← BACK TO INDEX](#)